

Endpoint Forensics and Monitoring

Velociraptor | Splunk Enterprise

Tahmid Abtahee | Cybersecurity Portfolio | April 2026

Project Overview

This project demonstrates hands-on endpoint detection and response (EDR) skills using two industry-standard tools: Velociraptor for live endpoint forensics and artifact collection, and Splunk Enterprise for log ingestion and SPL-based threat hunting. The environment consisted of a local Windows 10 host running the Velociraptor server, a remote AWS EC2 instance running Windows Server 2022 acting as the managed endpoint, and a Splunk instance receiving forwarded Windows event logs via the Universal Forwarder.

Work was split across four areas: reviewing Windows Security event logs natively in Event Viewer, deploying and enrolling an endpoint into Velociraptor, running artifact collections and VQL notebook queries, and building progressive SPL queries in Splunk to hunt for PowerShell execution activity and process creation events.

Velociraptor Version	0.75.6 (commit 0e657d636)
Velociraptor Server	Taholaptop (Windows 10, localhost:8889)
Managed Endpoint	EC2AMAZ-NGPKETK / EC2AMAZ-5PNITSP — Windows Server 2022 Datacenter 21H2
Endpoint Region	AWS us-west-2 (Oregon)
Splunk Version	Enterprise 10.2.2 (localhost:8000)
Splunk Forwarder Target	10.10.0.146:9997

Part 1: Windows Event Log Analysis

Before deploying any tooling, the Windows Event Viewer was used to examine the Security log directly on the local host. The Security log contained 30,768 recorded events at the time of review, providing a baseline understanding of the telemetry that EDR and SIEM platforms ingest.

The top event in the log was Event ID 4672, logged at 3/30/2026 10:55:39 PM. This event fires when special privileges are assigned to a new logon session. The detail pane shows the subject as NT AUTHORITY\SYSTEM (Logon ID: 0x3E7) on computer Taholaptop. Below it, Event ID 4624 (Successful Logon) and a high volume of Event ID 5379 (Credential Manager reads) are visible, all clustered at system startup. Recognizing the difference between expected startup noise and anomalous privilege assignments is a core SOC analyst skill, and 4672 is particularly

relevant when it appears for interactive or non-system accounts, which can indicate privilege escalation.

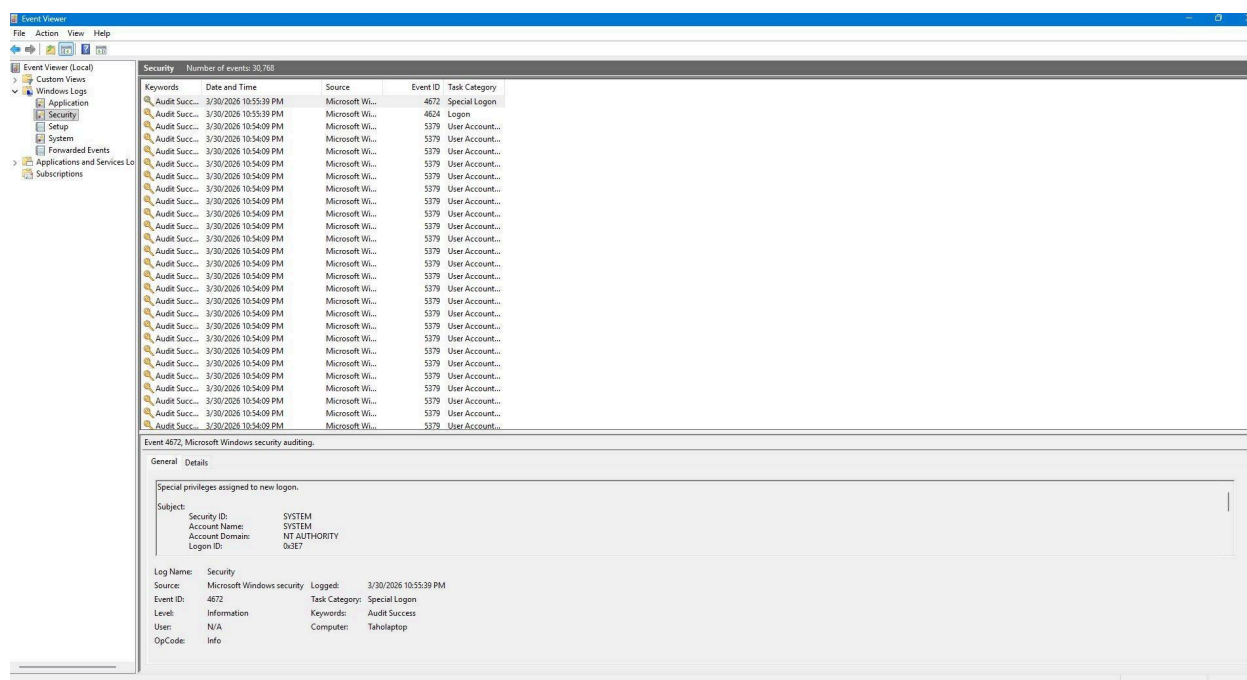


Figure 1: Windows Event Viewer Security log — Event ID 4672 (Special Logon) selected, showing SYSTEM account privilege assignment on Taholaptop

Events of Interest

- **Event ID 4672 — Special Logon:** Special privileges assigned to a new logon session. High-value for detecting privilege escalation when observed on non-system accounts.
- **Event ID 4624 — Successful Logon:** Baseline authentication event. Critical for lateral movement detection and unauthorized access investigations.
- **Event ID 5379 — Credential Manager Read:** Burst of reads at startup is normal. Unexpected bursts from unusual processes can indicate credential harvesting.

Part 2: Velociraptor Deployment and Client Enrollment

Server Setup and Dashboard

Velociraptor was deployed in self-signed certificate mode on the local host, with the GUI listening on localhost:8889 and the client comms endpoint on localhost:9999. Once initialized, the dashboard confirmed a single active organization (root) with the client configuration file (client.root.config.yaml) ready for download. Disk space on the C: drive was reported as 38 GB total, 15 GB free (39% available).

Current Orgs

Name	OrgId	ClientConfig
<root>	root	client.root.config.yaml

Disk Space

DeviceID	Description	VolumeName	VolumeSerialNumber	Size	FreeSpace	Free%
C:	Local Fixed Disk		1CE96F8A	38 GB	15 GB	39

Figure 2: Velociraptor server dashboard — root org created and C: drive disk space confirmed

Endpoint Enrollment

The client configuration YAML was downloaded from the dashboard and deployed to the AWS EC2 endpoint. The file specified the server URL (<https://localhost:9999/>) and embedded the CA certificate for mutual TLS. The Velociraptor client binary was launched on the endpoint and enrolled successfully. The client overview page confirmed a live green Connected status for EC2AMAZ-NGPKETK, with full metadata populated including OS release, architecture, MAC address, and first/last seen timestamps.

all

EC2AMAZ-NGPKETK.us-west-2.compute.internal Connected

Interrogate VFS Collected

Overview VQL Drilldown Shell

Client ID	2c1cda104cbea9f7c
Agent Version	0.75.6
Agent Build Time	2025-12-29T14:18:22Z
First Seen At	2025-04-03T14:35:17Z
Last Seen At	2025-04-03T17:03:52.971Z
Last Seen IP	10.1.1.43881
UUID	
Operating System	Windows
Hostnames	EC2AMAZ-NGPKETK
OS	EC2AMAZ-NGPKETK.us-west-2.compute.internal
Release	Microsoft Windows Server 2022 Datacenter21H2
Architecture	x64
MAC Addresses	82:05:98:84:ad:98

Client Metadata

Figure 3: Enrolled endpoint EC2AMAZ-NGPKETK showing Connected status, agent version 0.75.6, and full host metadata

Part 3: Artifact Collection and VQL Analysis

Hunt: Windows.System.Pslist

A hunt was created targeting all enrolled clients using the Windows.System.Pslist artifact, which enumerates running processes along with their PID, PPID, executable path, command-line arguments, username, and Authenticode signature data. The hunt completed in 1 second against the single enrolled endpoint, returning 73 process rows. The raw JSON results file was downloaded and opened to confirm the full process inventory, showing standard Windows Server processes including Registry, smss.exe, csrss.exe, wininit.exe, winlogon.exe, services.exe, lsass.exe, svchost.exe, and both Velociraptor agent processes confirming live agent activity.



Figure 4: Windows.System.Pslist hunt results — raw JSON output showing 73 processes collected from EC2AMAZ-NGPKETK in 1 second

VQL Notebook: Process Hierarchy Query

The Velociraptor Notebook was used to run a live VQL query directly against the connected endpoint, mapping processes to their parent processes:

```
SELECT Pid, Name, ParentName FROM pslist()
```

The query returned 75 rows and rendered a structured table showing the full Windows process tree: System (PID 4) as the root, Registry and smss.exe as its children, wininit.exe spawning services.exe and lsass.exe, and services.exe spawning multiple svchost.exe instances. This parent-child lineage mapping is foundational to detecting process injection, LOLBIN abuse, and unusual parent-child relationships that indicate malicious execution.

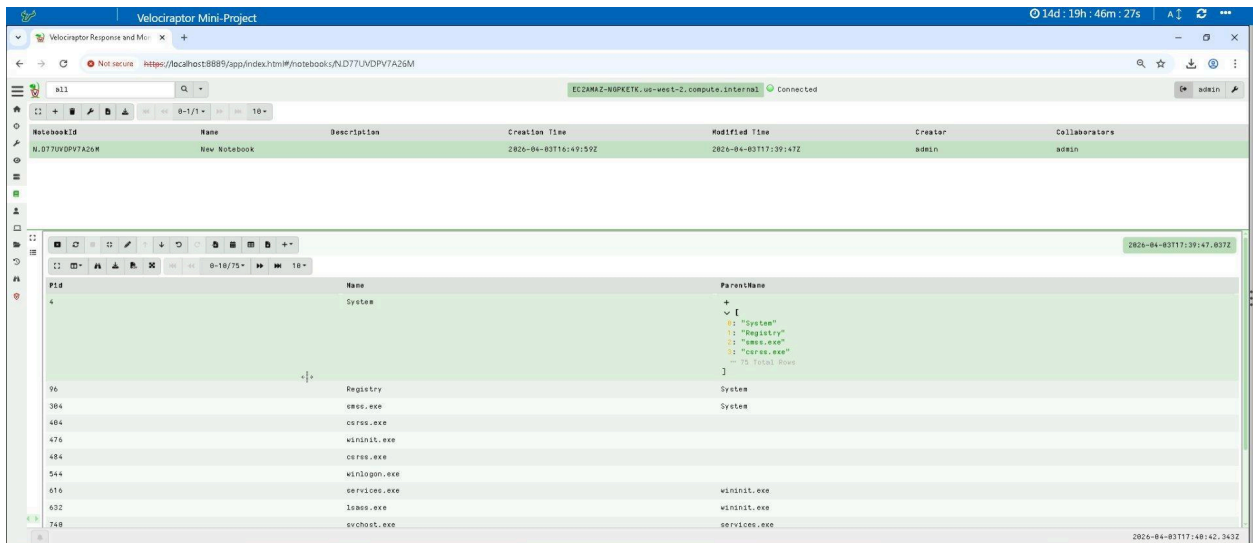


Figure 5: VQL Notebook pslist query — 75-row process hierarchy showing Pid, Name, and ParentName for the enrolled endpoint

Part 4: Splunk — Log Ingestion and Threat Hunting

Universal Forwarder Setup and Data Verification

The Splunk Universal Forwarder was installed on the EC2 endpoint and configured to forward Windows event logs to the Splunk indexer at 10.10.0.146:9997. After confirming the forwarder was running, a broad search was executed to verify data was flowing:

```
(index=* OR index=*)
```

The search returned 8,852 events in a 15-minute window (4/5/26 6:50 PM to 7:05 PM), confirming successful ingestion from host EC2AMAZ-5PNITSP. Selected fields showed 34 distinct sources and 25 source types. Event data visible in the results included Splunk audit trail entries (sourcetype=audittrail) and internal resource usage metrics (sourcetype=splunk_resource_usage), confirming the pipeline was live end to end.

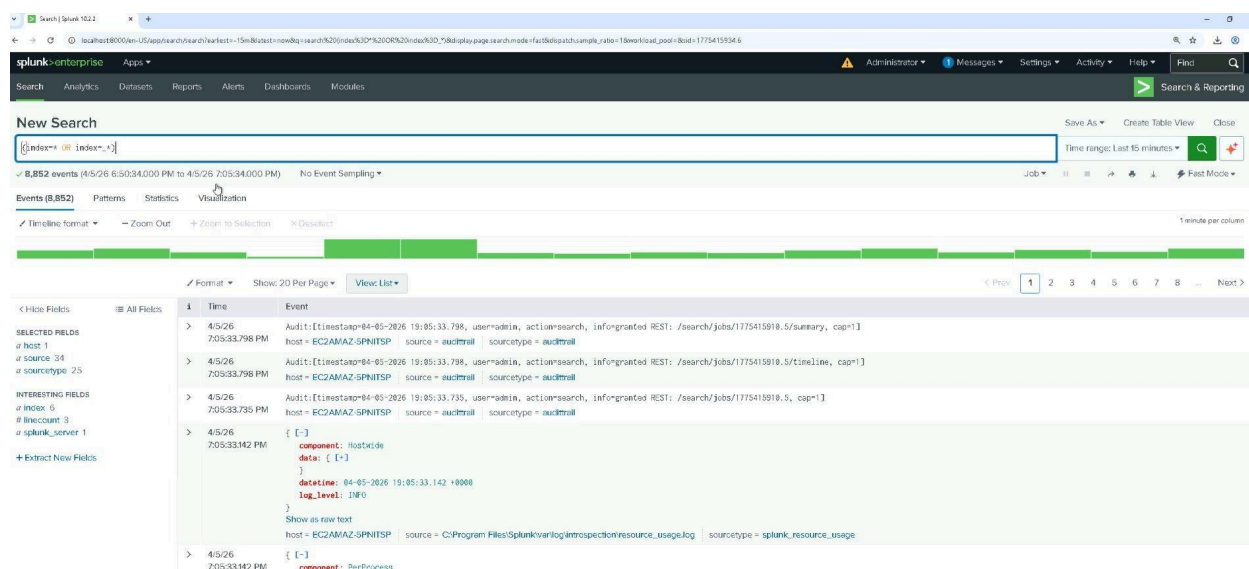


Figure 6: Splunk Search — broad index query returning 8,852 events, confirming log ingestion from EC2AMAZ-5PNITSP

PowerShell Script Block Logging — Event ID 4104

PowerShell Event ID 4104 captures the full text of every executed script block, making it one of the most valuable log sources for detecting obfuscated commands, download cradles, and living-off-the-land techniques. The following SPL query was built to extract the actual commands from the raw event text:

```

sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational"
EventCode=4104

| rex field=_raw "(?ms)Message=Creating Scriptblock text \\\(1 of
1\\):\\s*(?<cmd>.*)\\s*ScriptBlock ID:"

| stats count by cmd

| sort - count

```

The query matched 49 events and extracted 10 distinct commands (Statistics 10). Results visible in the output included Get-VM | Select-Object Name, Id, State (highest frequency), \$global:?, Set-StrictMode internal runtime blocks, an interactive prompt function, and three Splunk Universal Forwarder administrative commands run via PowerShell: bttool inputs list --debug, list forward-server, and list monitor. The presence of Splunk CLI commands in the script block log confirmed that PowerShell was actively used to manage the forwarder installation, and that script block logging was capturing that activity in real time.

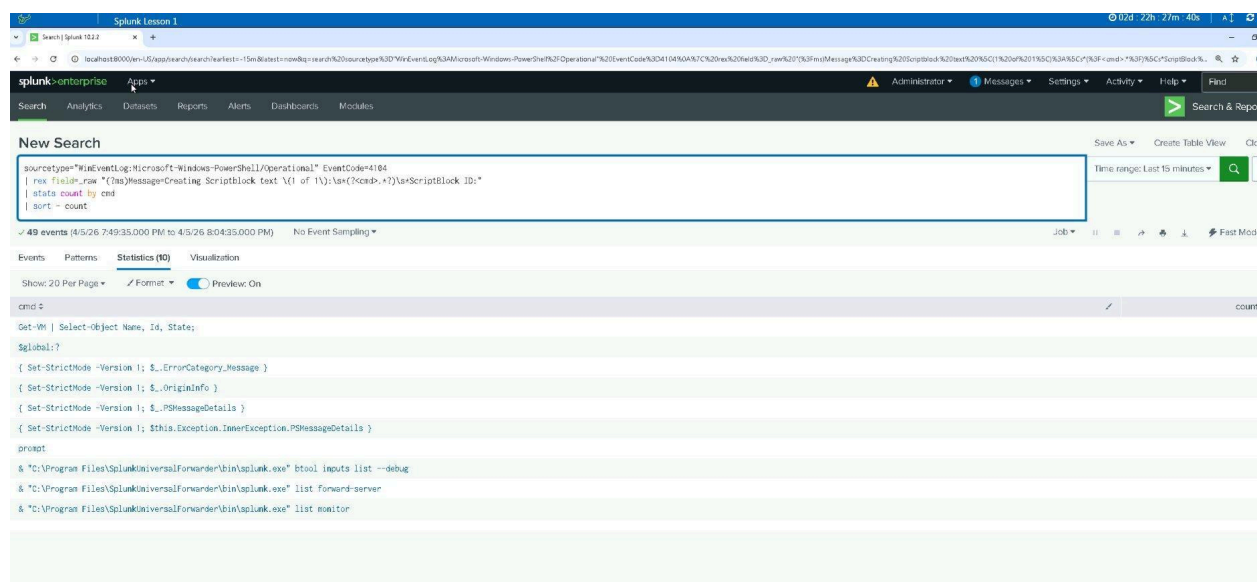


Figure 7: SPL rex extraction query — 49 events matched, 10 unique script block commands extracted including Get-VM enumeration and Splunk forwarder CLI calls

Process Creation Events — Event ID 4688

Event ID 4688 logs every process creation event and includes the new process name when command-line auditing is enabled. A timechart query was used to visualize process spawn activity over time broken down by process name:

```

(index=* OR index=_) EventCode=4688
| timechart count by New_Process_Name

```


The query returned 110 events over 15 minutes and produced a stacked column chart across 91 unique process names. The chart shows a clear burst of Splunk internal worker processes between 8:20 PM and 8:24 PM, including splunk-MonitorNoHandle.exe, splunk-netmon.exe, splunk-powershell.exe, splunk-regmon.exe, splunk-admon.exe, splunk-optimize.exe, and python313.exe. This burst pattern directly corresponds to the Splunk forwarder being configured and started during the lab session, providing a precise behavioral timeline of administrative actions on the endpoint.

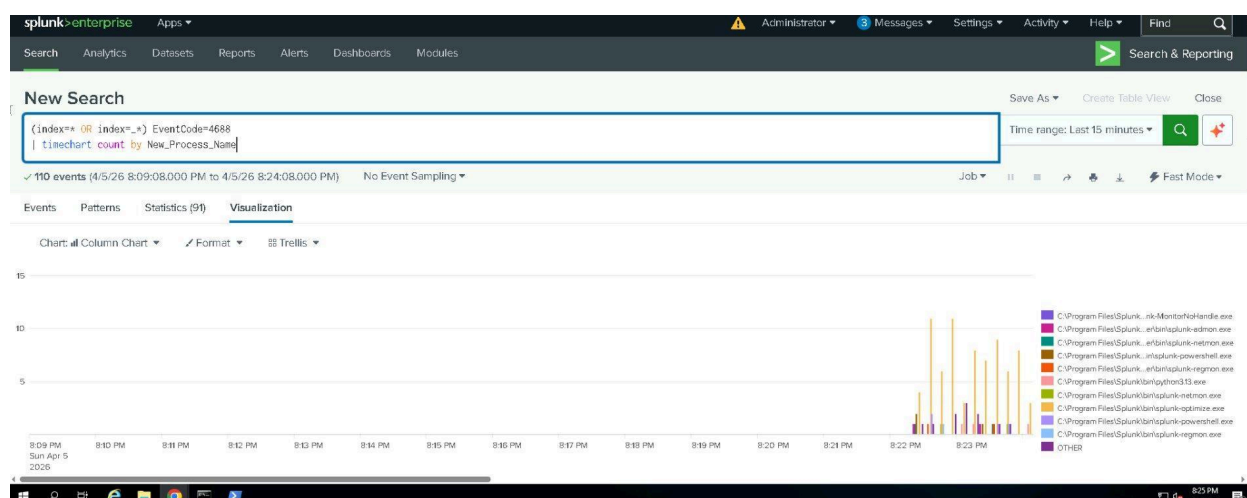


Figure 8: SPL timechart — EventCode=4688 process creation events visualized by New_Process_Name, showing Splunk worker process burst between 8:20–8:24 PM

Skills Demonstrated

- Velociraptor server deployment, organization setup, and self-signed TLS configuration
- Remote endpoint enrollment via client YAML config and live connectivity verification
- Artifact browsing and VQL artifact collection using Windows.System.Pslist and Windows.System.PowerShell
- Hunt creation, execution, and results download in Velociraptor
- Process enumeration and parent-child hierarchy analysis using VQL Notebooks
- Splunk Universal Forwarder installation and indexer registration via CLI
- SPL query development: filtering, rex field extraction, stats aggregation, timechart visualization
- PowerShell script block log analysis (Event ID 4104) for real-time command visibility
- Process creation event analysis (Event ID 4688) for behavioral baselining and timeline reconstruction
- Windows Security Event Log analysis including Event IDs 4624, 4672, and 5379

Tools and Technologies

Velociraptor	v0.75.6 — VQL-based EDR platform for live endpoint forensics and artifact collection
Splunk Enterprise	v10.2.2 — SIEM for log aggregation, SPL search, and visualization
Splunk Universal Forwarder	Windows agent for forwarding event logs to the Splunk indexer
Windows Event Viewer	Native Windows tool for Security, System, and Application log review
AWS EC2	Windows Server 2022 Datacenter 21H2 used as the remote managed endpoint
VQL	Velociraptor Query Language — SQL-like language for forensic artifact queries
SPL	Splunk Processing Language — used for all search, field extraction, and visualization
PowerShell	Used for client deployment commands and Splunk forwarder configuration

All activity performed in a personal lab environment. April 2026.